Práctica 5: Árboles de Expresiones

Construir, evaluar y recorrer un árbol

AED I — Bloque II

2 h en clase + ~1 h en casa · Python

Algoritmia y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

¿Para qué sirve esto?

¿Quién ha escrito alguna vez `=(A1+A2)*2` en una hoja de cálculo?

- Compiladores e intérpretes: Python convierte tu código en un árbol antes de ejecutarlo
- Hojas de cálculo: cada fórmula es un árbol de expresión
- Calculadoras y la notación polaca inversa
- Álgebra simbólica: Wolfram Alpha, SymPy (derivar, simplificar)

¿Qué vamos a hacer?

1. Repasamos los árboles binarios y sus recorridos, y vemos qué es un árbol de expresión
2. Construimos el árbol a partir de la notación postfija con una **pila de nodos**
3. Lo evaluamos y lo escribimos en las tres notaciones (los tres recorridos)
4. Calculamos sus métricas: altura, hojas (operandos) y nodos internos (operadores)
5. Cerramos con un test individual sobre lo trabajado hoy

Objetivos de la práctica

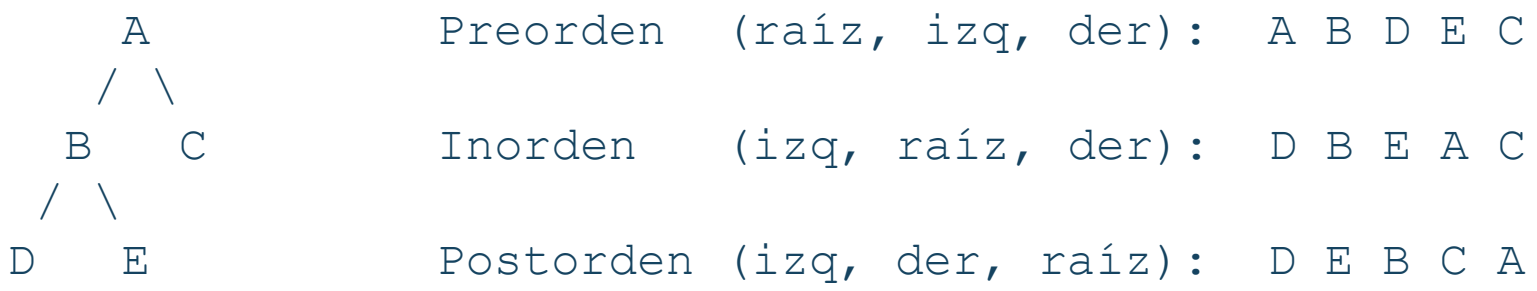
1. Representar una expresión aritmética con un árbol binario enlazado.
2. Construir un árbol a partir de su notación postfija usando una pila de nodos.
3. Entender para qué sirve cada recorrido: preorden, inorden y postorden.
4. Escribir algoritmos recursivos sobre árboles con el patrón **método público + auxiliar privado**.
5. Calcular métricas (altura, hojas, nodos internos) y analizar su complejidad.

Flujo de la sesión

Fase	Duración	Actividad
Motivación y repaso	10 min	Ejemplos reales; árboles binarios y recorridos
Ejercicio 1: Construir desde notación postfija	25 min	Pila de nodos + tests
Ejercicio 2: Evaluar	20 min	Recorrido postorden, con variables
Ejercicio 3: Las tres notaciones	20 min	Preorden, inorden y postorden
Ejercicio 4: Métricas	15 min	Altura, hojas, operadores (se termina en casa)
Cierre y test individual	10 min	Qué queda para casa + test

Repaso: árbol binario y recorridos

Cada nodo guarda un **dato** y dos punteros: `izquierdo` y `derecho`. Una **hoja** tiene los dos a `None`.

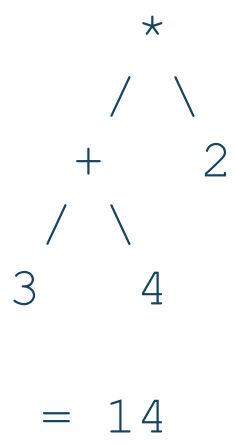


Altura: árbol vacío = −1, hoja = 0, altura(n) = 1 + max(altura(izq), altura(der)). Aquí: 2.

¿Qué es un árbol de expresión?

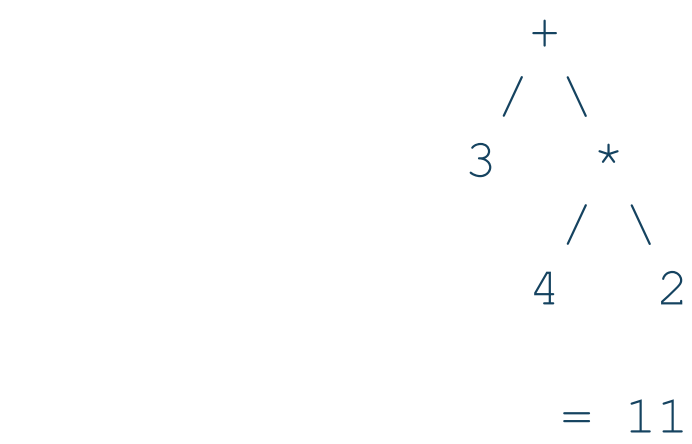
Hojas = operandos (números). **Nodos internos** = operadores binarios (`+ - * /`).

`(3 + 4) * 2`



`= 14`

`3 + 4 * 2`

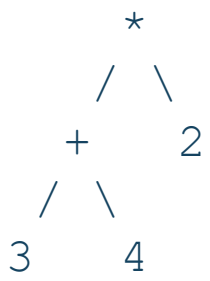


`= 11`

> **No hay paréntesis:** la forma del árbol fija el orden de las operaciones. Lo que está más abajo se calcula antes.

Tres notaciones = tres recorridos

Notación	(3 + 4) * 2	Recorrido
Prefija (polaca)	`* + 3 4 2`	Preorden: raíz → izq → der
Infija	`((3 + 4) * 2)`	Inorden: izq → raíz → der (+ paréntesis)
Postfija (polaca inversa)	`3 4 + 2 *`	Postorden: izq → der → raíz



La postfija se evalúa con **una sola pila** y sin paréntesis: así funcionan la JVM, el bytecode de Python o las calculadoras HP.

Cómo trabajamos: plantilla y tests

Completa `python/arbol_expresiones.py` en orden y pasa los tests de cada ejercicio:

```
cd python
python3 -m unittest -v test_arbol_expresiones.PruebasConstruccion
```

```
PruebasConstruccion   construir
PruebasEvaluacion     evaluar
PruebasNotaciones     prefija, infija, postfija
PruebasMetricas       altura, num_hojas            (num_nodos ya está hecho)
PruebasAmpliacion     infija_minima, por_niveles, construir_prefija   (opcional)
```

Ya hecho en `utilidades.py`: `dibujar(arbol)` e `infija_a_postfija("(3 + 4) * 2")`

Para ver tu árbol: `print(dibujar(ArbolExpresion("3 4 + 2 *")))`

Ejercicio 1: Construir desde postfija (1)

```
ArbolExpresion("3 4 + 2 *")        # el constructor llama a construir
def construir(self, postfija):      # TODO: devuelve la RAÍZ (un Nodo)
```

Recorremos `postfija.split()` con una **pila de NODOS** (`list`: `append` / `pop`):

- **Operando** → apilamos un nodo hoja
- **Operador** → desapilamos dos nodos, creamos el nodo operador con ellos como hijos y lo apilamos
- **Fin** → debe quedar exactamente un nodo en la pila: la raíz

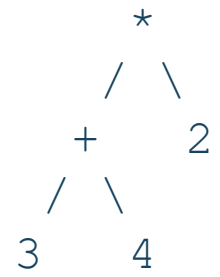
> **¡Cuidado!** El primero que sale de la pila es el hijo **DERECHO**: `8 2 -` debe valer 6, no −6.

Lanza `ValueError` si faltan o sobran operandos, si la entrada está vacía o si hay un token desconocido.

Ejercicio 1: Construir desde postfija (2)

Traza de `3 4 + 2 *`:

Token	Acción	Pila (fondo → cima)
`3`	apilar hoja	`[3]`
`4`	apilar hoja	`[3, 4]`
`+`	der = 4, izq = 3 → nodo +	`[(3+4)]`
`2`	apilar hoja	`[(3+4), 2]`
`*`	der = 2, izq = (3+4) → nodo *	`[((3+4)*2)]`



Fin: un solo elemento en la pila → **es la raíz**.

¿Cuánto cuesta? **O(n)**: cada token supone un número constante de operaciones de pila.

Ejercicio 2: Evaluar

```
def evaluar(self):    # TODO
```

- **Hoja**: su valor es el número → `int(nodo.valor)`
- **Operador**: evaluar el izquierdo, evaluar el derecho y aplicar `+ - * /`
- Mismo patrón que `num_nodos`: método público + auxiliar recursivo

```
ArbolExpresion("3 4 + 2 *").evaluar()     # 14
ArbolExpresion("7 2 /").evaluar()         # 3.5
ArbolExpresion().evaluar()                # ValueError: árbol vacío
```

Primero los hijos y después la raíz: ¿qué recorrido es?

Ejercicio 3: Las tres notaciones

```
def prefija(self):
    resultado = []
    self._preorden(self.raiz, resultado)
    return " ".join(resultado)

def _preorden(self, nodo, resultado):
    if nodo is None:
        return
    ...   # ¿en qué orden visitas
          # raíz, izq y der?
```

Método	(3 + 4) * 2	vacío
`prefija()`	`* + 3 4 2`	`""`
`postfija()`	`3 4 + 2 *`	`""`
`infija()`	`((3 + 4) * 2)`	`""`

Un único espacio entre tokens, sin espacios al principio ni al final. En infija, cada operación va entre paréntesis y una hoja va sin ellos (`x` → `"x"`).

Ejercicio 4: Métricas

```
def num_nodos(self):      # HECHO: ejemplo del patrón (diapositiva siguiente)
def altura(self):         # TODO: vacío -> -1, hoja -> 0
def num_hojas(self):      # TODO
def num_internos(self):  #TODO
```

```
      +          "2 3 * 4 +"
     / \
    *   4        altura = 2     num_nodos = 5     num_hojas = 3
   / \
  2   3
```

> **Árbol binario lleno** (cada nodo tiene 0 o 2 hijos): hojas = operadores + 1.

El patrón recursivo: público + auxiliar

Bien

```
def num_nodos(self):
    return self._num_nodos(self.raiz)

def _num_nodos(self, nodo):
    if nodo is None:
        return 0
    i = self._num_nodos(nodo.izquierdo)
    d = self._num_nodos(nodo.derecho)
    return 1 + i + d
```

Mal `nodo=None`

```
def num_nodos(self, nodo=None):
    if nodo is None:
        nodo = self.raiz   # ¡¡!!
    ...
    self.num_nodos(nodo.izquierdo)
```

Con `nodo=None`, al llegar a un hijo vacío **se vuelve a empezar por la raíz**: recursión infinita o resultado incorrecto.

Ampliación (opcional): otras operaciones

`infija_minima()` — solo los paréntesis necesarios

```
((3 + 4) * 2)  ->  (3 + 4) * 2          (8 - (4 - 1))  ->  8 - (4 - 1)
```

`por_niveles()` — recorrido en anchura, con una cola (collections.deque)

```
(3 + 4) * 2  ->  "* + 2 3 4"
```

`construir_prefija(prefija)` — construir desde la prefija, sin pila: con recursión

```
"* + 3 4 2"  ->  el árbol de (3 + 4) * 2
```

Cierre y actividad para casa

Si no lo has terminado, en casa

- Ejercicio 4 (métricas) → `PruebasMetricas`
- `RESPUESTAS.md`: tabla de costes y tres preguntas

Ampliación (opcional)

- `infija_minima()`, `por_niveles()`, `construir_prefija()`

Ahora: test individual